Matrícula Escalável

Sistema distribuído de inscrição em disciplinas resiliente a picos de acesso

Projeto 02 · Documentação inicial

Engenharia de Sistemas Distribuídos · UFPB 2026.2

Daniel Victor Carneiro Brandão da Costa · Davi Nasiasene Amorim · Ian Rocha Bittencourt

O pico: milhares de alunos, poucas vagas

Sistemas monolíticos, no pico do início do período:



Degradam
O sistema fica lento justamente quando todos acessam ao mesmo tempo



Caem
O pico derruba o serviço e ninguém consegue se matricular



Aceitam além da capacidade
Overbooking e matrículas duplicadas com retentativas e falhas parciais

5.000

alunos em 1 minuto

200

vagas em disputa

Cenário-alvo da proposta (valores preliminares)

Cinco critérios de sucesso, todos mensuráveis

1

0

matrículas além da capacidade ou duplicadas

5.000 usuários em 1 minuto disputando 200 vagas

2

< 200 ms

p95 da consulta de vagas

sob a mesma carga do critério 1

3

≥ 95%

de sucesso das matrículas aceitas

sob sobrecarga, comparando com e sem load shedding

4

< 30 s

para se recuperar

da queda de uma instância de Inscrição no pico, sem perder nem duplicar

5

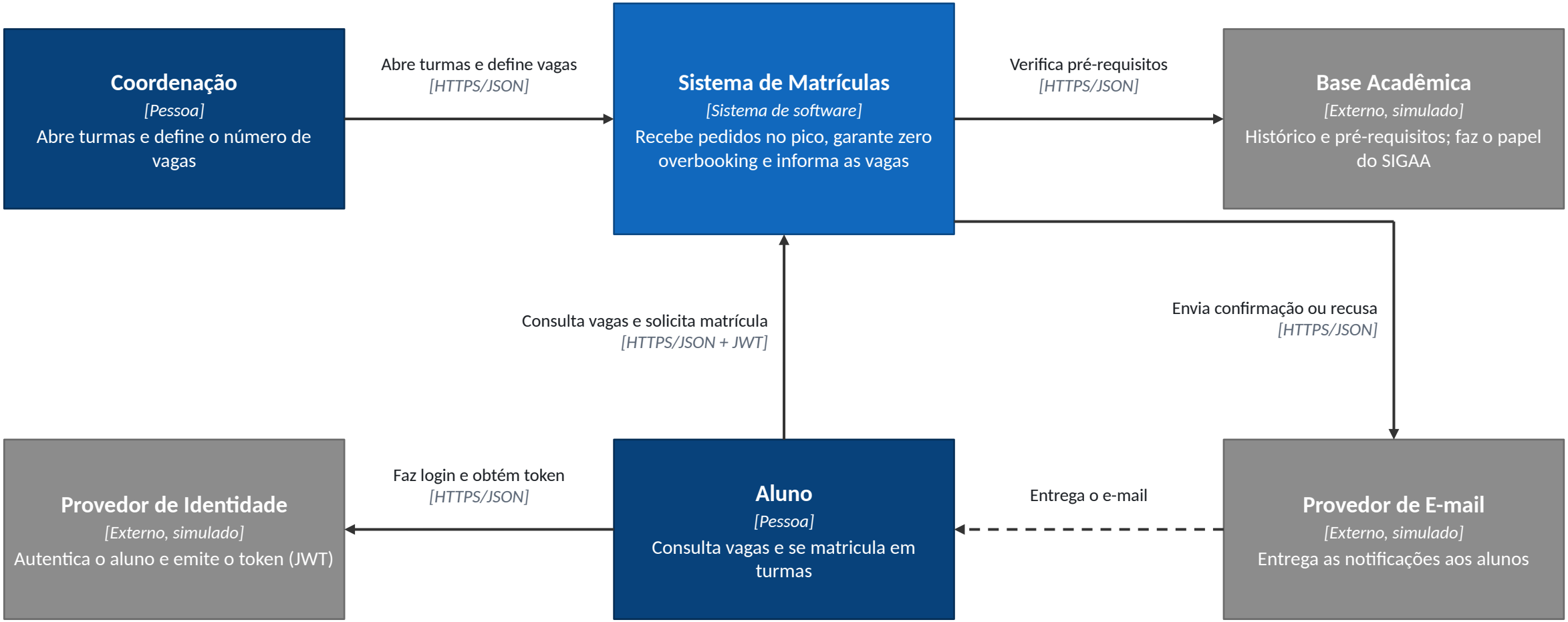
≥ 95%

de sucesso com a Notificação fora do ar

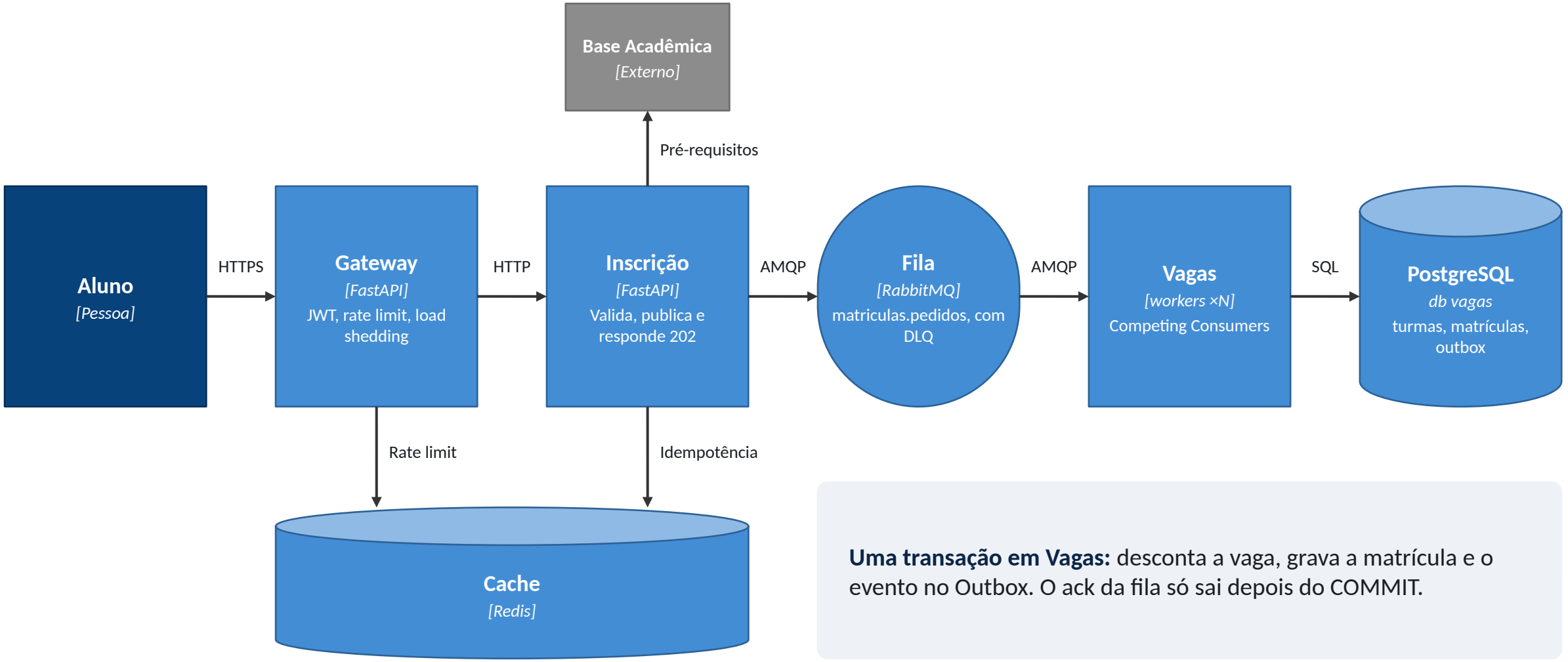
e pendências entregues após a recuperação

Cada padrão arquitetural valida pelo menos um destes critérios.

C4 nível 1: o sistema e seus vizinhos

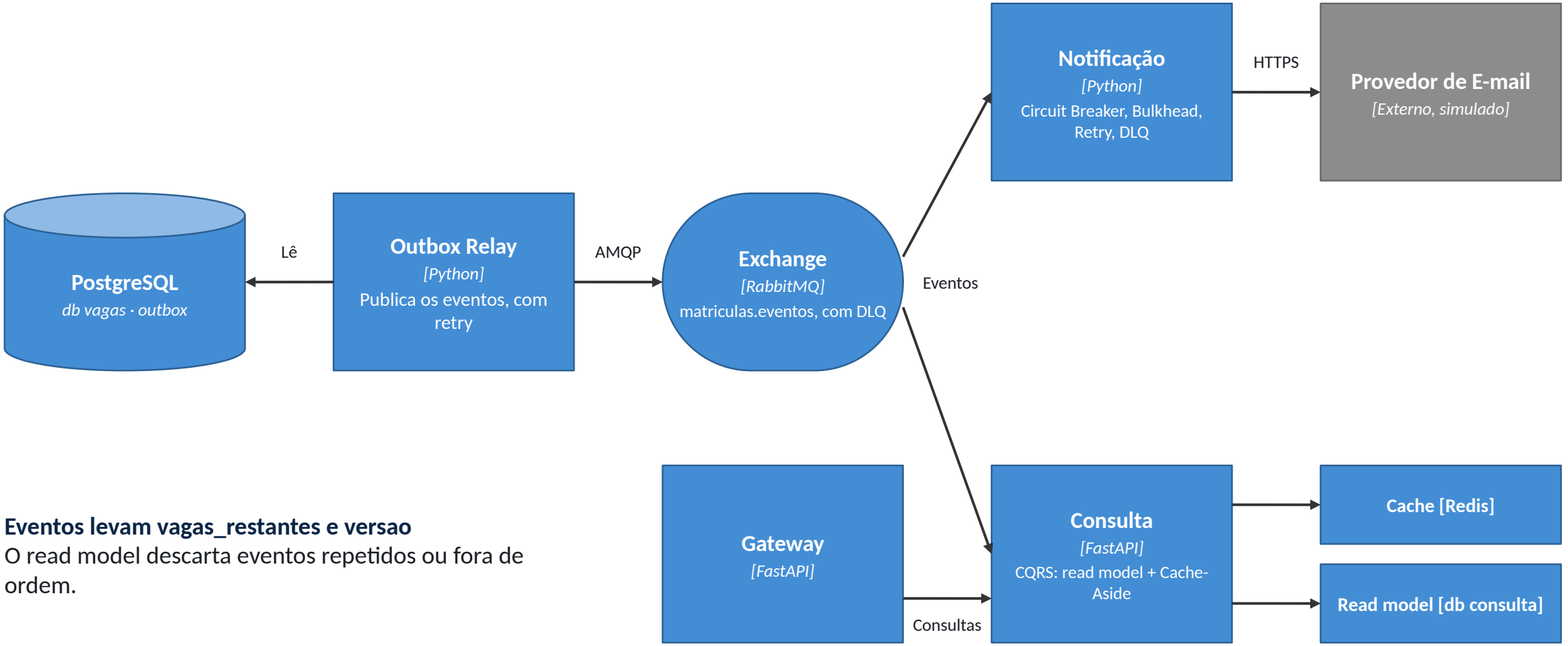


C4 nível 2: o caminho de um pedido de matrícula



Uma transação em Vagas: desconta a vaga, grava a matrícula e o evento no Outbox. O ack da fila só sai depois do COMMIT.

C4 nível 2: eventos e leitura



Eventos levam vagas_restantes e versao
O read model descarta eventos repetidos ou fora de ordem.

Seis padrões, cada um ligado a um critério



ESCALABILIDADE
Critérios 1, 4

Queue / Competing Consumers

Absorve o pico e escala o processamento

Fila + Vagas (workers ×N)



DESEMPENHO
Critérios 3

Rate Limit + Load Shedding

Rejeita o excedente com 429/503

Gateway (Redis)



DESEMPENHO
Critérios 2

CQRS + Cache-Aside

Leitura rápida, separada da escrita

Consulta + read model + Redis



CONFIABILIDADE
Critérios 1, 4

Idempotência

Clique repetido conta uma vez só

Inscrição (Redis) + UNIQUE



CONFIABILIDADE
Critérios 4, 5

Transactional Outbox + Retry/DLQ

Matrícula e evento na mesma transação

Vagas + Outbox Relay + DLQs



CONFIABILIDADE
Critérios 3, 5

Circuit Breaker + Bulkhead

Falha de um não derruba os demais

Notificação, Gateway

Padrões considerados e descartados

SAGA / Orchestration	Vaga, matrícula e outbox ficam numa única transação em Vagas (ADR-001): não há transação distribuída
Inbox	A idempotência dos consumidores já vem da versao no read model (ADR-002) e da restrição UNIQUE
Request Hedging	Duplicaria requisições justamente no pico, aumentando a carga
Service Mesh	Complexidade de infraestrutura desproporcional para 6 containers
Event Sourcing	O estado atual de vagas e matrículas atende ao domínio; histórico completo não é requisito

ADR-001: Vagas é o único dono da escrita

Inscrição só publica

Valida pré-requisito e idempotência, publica na fila e responde 202. Não escreve matrícula.

Uma transação em Vagas

Desconta a vaga, grava a matrícula e o evento no Outbox. Sem vaga, grava matricula_recusada.

Reentrega segura

ack só depois do COMMIT. Na reentrega, UNIQUE (aluno_id, turma_id) barra a duplicata.

```
BEGIN;
UPDATE turmas
   SET vagas = vagas - 1,
       versao = versao + 1
 WHERE id = :turma_id AND vagas > 0
RETURNING vagas, versao;  -- 0 linhas: sem vaga

INSERT INTO matriculas
  (pedido_id, aluno_id, turma_id)
VALUES (...);  -- UNIQUE (aluno_id, turma_id)

INSERT INTO outbox (tipo, payload)
VALUES ('matricula_confirmada', {...});
COMMIT;
```

Custo aceito: resposta assíncrona (202), linha quente nas turmas disputadas e ordem aproximada entre workers.

ADR-002: read model próprio da Consulta

Database consulta, atualizado pelos eventos. Só aplica o evento se a versao for maior:

```
INSERT INTO turmas_leitura
  (turma_id, vagas_restantes, versao)
VALUES (:turma_id, :vagas_restantes, :versao)
ON CONFLICT (turma_id) DO UPDATE
  SET vagas_restantes = EXCLUDED.vagas_restantes,
      versao = EXCLUDED.versao
WHERE turmas_leitura.versao < EXCLUDED.versao;
```

Consistência eventual: a tela pode mostrar uma vaga já ocupada, mas quem decide é a transação de Vagas, então não há overbooking.

Contra cache stampede



Lock de recomputação
Só a primeira requisição vai à fonte (SET NX); as demais leem do cache



TTL com jitter
As chaves não expiram todas juntas



Aquecimento
Turmas carregadas no Redis antes da abertura

Stack: uma linguagem, tudo código aberto e local

Python 3.12 + FastAPI
Todos os serviços, assíncronos

RabbitMQ 4
Fila de pedidos, exchange de eventos, DLQs

PostgreSQL 16
db vagas e db consulta

Redis 7
Rate limit, idempotência, cache

JWT HS256
Validado localmente no Gateway

Docker Compose
Sobe tudo com um comando

GitHub Actions
Lint, testes e Compose a cada push

k6
Testes de carga dos critérios 1 a 5

Prometheus + Grafana
Métricas RED e dashboards

OpenTelemetry + Jaeger
Trace de um pedido ponta a ponta

Ruff + pytest
Lint, formatação e testes

Mermaid
Diagramas C4 como código

Plano de testes

Carga progressiva

Fila, rate limit e load shedding

Mede: p95 e taxa de sucesso

Concorrência

Muitas requisições para a mesma vaga

Mede: zero overbooking

Resiliência

Queda de Inscrição, do broker e da Notificação

Mede: nada perdido nem duplicado

Integração

Fluxo completo, Outbox, retry e DLQ

Mede: eventos entregues

k6 gera a carga; Prometheus, Grafana e Jaeger mostram o que aconteceu dentro do sistema.

Próximos passos



09/10

Projeto 01

Grupo e tema

06/11

Projeto 02 · hoje

Documentação inicial: C4, padrões, ADRs, stack

11/12

Projeto 03

Implementação, resultados dos testes e apresentação

- Implementar os serviços sobre o esqueleto já pronto (Compose, CI, healthchecks)
- Rodar os testes de carga e resiliência e coletar métricas
- Fechar os ADRs e publicar os resultados no README